NAME : BISMA FATIMA

CLASS : CS A

ROLL NO: F24605036

COURSE : OBJECT ORIENTED PROGRAMMING

COURSE CODE : CS-123

LECTURER : MISS TAYYABA KALSOOM

LAB TASK 10

Question 1

You are hired by a smart city project to develop a basic model for **smart traffic lights**.

PROGRAM

```
// Abstract Class
abstract class TrafficLight {
    protected String state = "Red";

    // Abstract Methods
    abstract void changeLight();
    abstract String getCurrentState();
}

// Smart Traffic Light
class SmartTrafficLight extends TrafficLight {
    @Override
    void changeLight() {
        if (state.equals("Red")) {
            state = "Green";
        } else if (state.equals("Green")) {
            state = "Yellow";
        } else {
            state = "Red";
        }
    }

    @Override
    String getCurrentState() {
        return state;
    }
}

// Manual Traffic Light
class ManualTrafficLight extends TrafficLight {
    @Override
    void changeLight() {
        if (state.equals("Red")) {
            state = "Green";
        } else if (state.equals("Green")) {
            state = "Yellow";
        } else {
            state = "Red";
        }
    }

    @Override
    String getCurrentState() {
        return state;
```

```
    }
}

// Main Method
public class Main {
    public static void main(String[] args) {
        SmartTrafficLight smartLight = new SmartTrafficLight();
        ManualTrafficLight manualLight = new ManualTrafficLight();

        smartLight.changeLight();
        manualLight.changeLight();

        System.out.println("Smart Traffic Light is " + smartLight.getCurrentState());
        System.out.println("Manual Traffic Light is " + manualLight.getCurrentState());
    }
}
```

OUTPUT

```
Smart Traffic Light is Green
Manual Traffic Light is Green
```

Question 2

You are building a **virtual art gallery system** where different types of artworks can be displayed and priced.

PROGRAM

```
// Abstract Class
abstract class Artwork {
    protected String title;
    protected double price;

    public Artwork(String title, double price) {
        this.title = title;
        this.price = price;
    }

    abstract void display();
    abstract double getPrice();
}

// Painting Class
class Painting extends Artwork {
    private String artist;

```

```
    public Painting(String title, double price, String artist) {
        super(title, price);
        this.artist = artist;
    }

    @Override
    void display() {
        System.out.println("Painting: " + title + " by " + artist);
    }

    @Override
    double getPrice() {
        return price;
    }
}

// Sculpture Class
class Sculpture extends Artwork {
    private String material;

    public Sculpture(String title, double price, String material) {
        super(title, price);
        this.material = material;
    }

    @Override
    void display() {
        System.out.println("Sculpture: " + title + " made of " + material);
    }

    @Override
    double getPrice() {
        return price;
    }
}

// Main Method
public class ArtGallery {
    public static void main(String[] args) {
        Painting painting = new Painting("Starry Night", 1500.0, "Vincent van Gogh");
        Sculpture sculpture = new Sculpture("The Thinker", 3000.0, "Bronze");

        painting.display();
        System.out.println("Price: $" + painting.getPrice());

        sculpture.display();
        System.out.println("Price: $" + sculpture.getPrice());
    }
}
```

OUTPUT

```
Painting: Starry Night by Vincent van Gogh
Price: $1500.0
Sculpture: The Thinker made of Bronze
Price: $3000.0
```

Question 3

You have been asked to design a simple program for a **space agency** to simulate different spacecraft.

PROGRAM

```
// Abstract Class
abstract class Spacecraft {
    protected String name;
    public Spacecraft(String name) {
        this.name = name;
    }
    abstract void launch();
    abstract void land();
}
// Cargo Ship Class
class CargoShip extends Spacecraft {
    public CargoShip(String name) {
        super(name);
    }
    @Override
    void launch() {
        System.out.println(name + " is launching with cargo.");
    }
    @Override
    void land() {
        System.out.println(name + " is landing with supplies.");
    }
}
// Passenger Ship Class
class PassengerShip extends Spacecraft {
    public PassengerShip(String name) {
        super(name);
    }
    @Override
    void launch() {
        System.out.println(name + " is launching with astronauts.");
    }
    @Override
    void land() {
```

```
        System.out.println(name + " is landing safely.");
    }
}
// Main Class
public class SpaceAgency {
    public static void main(String[] args) {
        CargoShip cargoShip = new CargoShip("Galaxy Freight");
        PassengerShip passengerShip = new PassengerShip("Star Voyager");
        cargoShip.launch();
        cargoShip.land();
        passengerShip.launch();
        passengerShip.land();
    }
}
```

OUTPUT

```
Galaxy Freight is launching with cargo.
Galaxy Freight is landing with supplies.
Star Voyager is launching with astronauts.
Star Voyager is landing safely.
```

Question 4

You are developing a **wildlife animal tracking system**.

PROGRAM

```
// Abstract class AnimalTracker
abstract class AnimalTracker {
    abstract void track();
}
// Subclass ElephantTracker
class ElephantTracker extends AnimalTracker {
    @Override
    void track() {
        System.out.println("Tracking an elephant... GPS coordinates received.");
    }
}
// Subclass TigerTracker
class TigerTracker extends AnimalTracker {
    @Override
    void track() {
        System.out.println("Tracking a tiger... Heat signature detected.");
    }
}
// Main class
public class WildlifeTracker {
```

```
    public static void main(String[] args) {
        AnimalTracker elephantTracker = new ElephantTracker();
        AnimalTracker tigerTracker = new TigerTracker();
        elephantTracker.track();
        tigerTracker.track();
    }
}
```

OUTPUT

```
Tracking an elephant... GPS coordinates received.
Tracking a tiger... Heat signature detected.
```

Question 5

You are working on a **home automation system** where smart appliances can be controlled.

PROGRAM

```
// Abstract class Appliance
abstract class Appliance {
    abstract void turnOn();
    abstract void turnOff();
}
// Subclass SmartLight
class SmartLight extends Appliance {
    @Override
    void turnOn() {
        System.out.println("Smart Light is now ON.");
    }
    @Override
    void turnOff() {
        System.out.println("Smart Light is now OFF.");
    }
}
// Subclass SmartFan
class SmartFan extends Appliance {
    @Override
    void turnOn() {
        System.out.println("Smart Fan is now ON.");
    }
    @Override
    void turnOff() {
        System.out.println("Smart Fan is now OFF.");
    }
}
// Main class
public class HomeAutomation {
```

```
    public static void main(String[] args) {
        Appliance smartLight = new SmartLight();
        Appliance smartFan = new SmartFan();
        smartLight.turnOn();
        smartLight.turnOff();
        smartFan.turnOn();
        smartFan.turnOff();
    }
}
```

OUTPUT

```
Smart Light is now ON.
Smart Light is now OFF.
Smart Fan is now ON.
Smart Fan is now OFF.
```

Question 6

A food delivery app needs a basic system to represent different **food items**. **Requirements:**

PROGRAM

```
// Abstract class FoodItem
abstract class FoodItem {
    abstract void prepare();
    abstract double getCost();
}
// Subclass Pizza
class Pizza extends FoodItem {
    @Override
    void prepare() {
        System.out.println("Preparing pizza... Baking dough, adding toppings, and cheese.");
    }
    @Override
    double getCost() {
        return 12.99; // Example cost in USD
    }
}
// Subclass Salad
class Salad extends FoodItem {
    @Override
    void prepare() {
        System.out.println("Preparing salad... Chopping vegetables and mixing dressing.");
    }
    @Override
    double getCost() {
        return 7.99; // Example cost in USD
```

```
    }
}
// Main class
public class FoodDeliveryApp {
    public static void main(String[] args) {
        FoodItem pizza = new Pizza();
        FoodItem salad = new Salad();
        pizza.prepare();
        System.out.println("Pizza cost: $" + pizza.getCost());
        salad.prepare();
        System.out.println("Salad cost: $" + salad.getCost());
    }
}
```

OUTPUT

```
Preparing pizza... Baking dough, adding toppings, and cheese.
Pizza cost: $12.99
Preparing salad... Chopping vegetables and mixing dressing.
Salad cost: $7.99
```

Question 7

Your university library wants to automate borrowing of different **library items**.

PROGRAM

```
// Abstract class LibraryItem
abstract class LibraryItem {
    abstract void borrow();
    abstract void returnItem();
}
// Subclass EBook
class EBook extends LibraryItem {
    @Override
    void borrow() {
        System.out.println("Borrowing eBook... Access granted.");
    }
    @Override
    void returnItem() {
        System.out.println("Returning eBook... Access revoked.");
    }
}
// Subclass AudioBook
class AudioBook extends LibraryItem {
    @Override
    void borrow() {
        System.out.println("Borrowing Audiobook... Audio playback unlocked.");
```

```
    }
    @Override
    void returnItem() {
        System.out.println("Returning Audiobook... Playback access removed.");
    }
}
// Main class
public class LibraryAutomation {
    public static void main(String[] args) {
        LibraryItem ebook = new EBook();
        LibraryItem audiobook = new AudioBook();

        ebook.borrow();
        ebook.returnItem();
        audiobook.borrow();
        audiobook.returnItem();
    }
}
```

OUTPUT

```
Borrowing eBook... Access granted.
Returning eBook... Access revoked.
Borrowing Audiobook... Audio playback unlocked.
Returning Audiobook... Playback access removed.
```

Question 8

A company is developing an **AI-based language translator**.

PROGRAM

```
// Abstract class Translator
abstract class Translator {
    abstract void translate(String text);
}
// Subclass EnglishToFrench
class EnglishToFrench extends Translator {
    @Override
    void translate(String text) {
        System.out.println("French Translation of '" + text + "': Bonjour! Comment ça va?");
    }
}
// Subclass EnglishToSpanish
class EnglishToSpanish extends Translator {
    @Override
    void translate(String text) {
        System.out.println("Spanish Translation of '" + text + "': ¡Hola! ¿Cómo estás?");
```

```
    }
}
// Main class
public class AITranslator {
    public static void main(String[] args) {
        Translator frenchTranslator = new EnglishToFrench();
        Translator spanishTranslator = new EnglishToSpanish();

        String sentence = "Hello! How are you?";
        frenchTranslator.translate(sentence);
        spanishTranslator.translate(sentence);
    }
}
```

OUTPUT

```
French Translation of 'Hello! How are you?': Bonjour! Comment ça va?
Spanish Translation of 'Hello! How are you?': ¡Hola! ¿Cómo estás?
```

Question 9

A robotics company is testing **autonomous delivery robots** for different environments.

PROGRAM

```
// Abstract class DeliveryBot
abstract class DeliveryBot {
    abstract void navigate();
    abstract void deliverPackage();
}
// Subclass UrbanBot
class UrbanBot extends DeliveryBot {
    @Override
    void navigate() {
        System.out.println("UrbanBot navigating through city streets using GPS and traffic data.");
    }
    @Override
    void deliverPackage() {
        System.out.println("UrbanBot delivering package to high-rise building doorstep.");
    }
}
// Subclass RuralBot
class RuralBot extends DeliveryBot {
    @Override
    void navigate() {
        System.out.println("RuralBot navigating through rough terrain using satellite imagery.");
    }
    @Override
```

```
    void deliverPackage() {
        System.out.println("RuralBot delivering package to a farmhouse mailbox.");
    }
}
// Main class
public class RoboticsCompany {
    public static void main(String[] args) {
        DeliveryBot urbanBot = new UrbanBot();
        DeliveryBot ruralBot = new RuralBot();

        urbanBot.navigate();
        urbanBot.deliverPackage();
        ruralBot.navigate();
        ruralBot.deliverPackage();
    }
}
```

OUTPUT

```
UrbanBot navigating through city streets using GPS and traffic data.
UrbanBot delivering package to high-rise building doorstep.
RuralBot navigating through rough terrain using satellite imagery.
RuralBot delivering package to a farmhouse mailbox.
```

Question 10

You are developing a simple **digital voting machine system**.

PROGRAM

```
// Abstract class VotingMachine
abstract class VotingMachine {
    abstract void castVote(String candidate);
    abstract void displayResults();
}
// Subclass StudentCouncilVoting
class StudentCouncilVoting extends VotingMachine {
    private int votesForBisma = 0;
    private int votesForAyesha = 0;
    @Override
    void castVote(String candidate) {
        if (candidate.equalsIgnoreCase("Bisma")) {
            votesForBisma++;
            System.out.println("Vote cast for Bisma.");
        } else if (candidate.equalsIgnoreCase("Ayesha")) {
            votesForAyesha++;
            System.out.println("Vote cast for Ayesha.");
        } else {
```

```
            System.out.println("Invalid candidate.");
        }
    }
    @Override
    void displayResults() {
        System.out.println("Student Council Voting Results:");
        System.out.println("Bisma: " + votesForBisma + " votes");
        System.out.println("Ayesha: " + votesForAyesha + " votes");
    }
}
// Subclass EmployeeOfTheMonthVoting
class EmployeeOfTheMonthVoting extends VotingMachine {
    private int votesForWania = 0;
    private int votesForMohsina = 0;
    private int votesForJaveia = 0;
    @Override
    void castVote(String candidate) {
        if (candidate.equalsIgnoreCase("Wania")) {
            votesForWania++;
            System.out.println("Vote cast for Wania.");
        } else if (candidate.equalsIgnoreCase("Mohsina")) {
            votesForMohsina++;
            System.out.println("Vote cast for Mohsina.");
        } else if (candidate.equalsIgnoreCase("Javeia")) {
            votesForJaveia++;
            System.out.println("Vote cast for Javeia.");
        } else {
            System.out.println("Invalid candidate.");
        }
    }
    @Override
    void displayResults() {
        System.out.println("Employee of the Month Voting Results:");
        System.out.println("Wania: " + votesForWania + " votes");
        System.out.println("Mohsina: " + votesForMohsina + " votes");
        System.out.println("Javeia: " + votesForJaveia + " votes");
    }
}
// Main class
public class DigitalVotingMachine {
    public static void main(String[] args) {
        VotingMachine studentVoting = new StudentCouncilVoting();
        VotingMachine employeeVoting = new EmployeeOfTheMonthVoting();
        // Simulating votes
        studentVoting.castVote("Bisma");
        studentVoting.castVote("Ayesha");
        studentVoting.castVote("Bisma");
        employeeVoting.castVote("Wania");
        employeeVoting.castVote("Mohsina");
        employeeVoting.castVote("Javeia");
        employeeVoting.castVote("Mohsina");
```

```
        // Displaying results
        studentVoting.displayResults();
        employeeVoting.displayResults();
    }
}
```

OUTPUT

```
Vote cast for Bisma.
Vote cast for Ayesha.
Vote cast for Bisma.
Vote cast for Wania.
Vote cast for Mohsina.
Vote cast for Javeia.
Vote cast for Mohsina.
Student Council Voting Results:
Bisma: 2 votes
Ayesha: 1 votes
Employee of the Month Voting Results:
Wania: 1 votes
Mohsina: 2 votes
Javeia: 1 votes
```